# MSAGLJS: Graph Layout, Edge Routing, and Tiling for the Web

**Anonymous author(s)**
Anonymous affiliation(s)

## Abstract

We present MSAGLJS, an open-source TypeScript library for graph layout, edge routing, and interactive visualization in web browsers, distributed as a set of NPM packages[1]. It provides force-directed, Sugiyama, and MDS layouts with routed edges, rendered using WebGL via the deck.gl framework.

We describe two main contributions. First, a *corridor routing* method that routes edges directly on the dual graph of a Constrained Delaunay Triangulation (CDT), producing for each edge the shortest path in its homotopy class. Each node's obstacle boundary defines *portal triangles*—free-space CDT triangles adjacent to the obstacle—that serve as entry and exit points in a *portal graph*. To accelerate the shortest-path queries on this graph, we preprocess it with Contraction Hierarchies and Hub Labeling, reducing per-edge query time from $O(T \log T)$ to $O(|L|)$, where $T$ is the number of free-space triangles and $|L|$ is the average hub-label size.

Second, a *tiling scheme* that enables map-like exploration of large graphs. The scheme builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, and simplifies edge routes at coarser levels. The WebGL renderer dynamically loads tiles based on the viewport, allowing smooth interaction with graphs of up to 10 000 nodes and 40 000 edges in a standard browser.

MSAGLJS is available at `https://github.com/microsoft/msagljs`.

**2012 ACM Subject Classification** Human-centered computing → Graph drawings; Theory of computation → Computational geometry

**Keywords and phrases** Graph visualization, edge routing, Constrained Delaunay Triangulation, contraction hierarchies, hub labeling, tiling, WebGL

**Category** Track 2, Long Paper

**Generative AI Declaration** Generative AI was used for editorial assistance in the preparation of this article.

## 1 Introduction

Visualizing graphs in web browsers presents unique challenges: the runtime is single-threaded, memory is limited, and users expect the smooth pan-and-zoom experience familiar from online maps. MSAGLJS[2] is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme for level-of-detail rendering.

MSAGLJS is consumed as a set of NPM packages and renders graphs using WebGL through the deck.gl framework [4]. It supports force-directed, Sugiyama (layered), and MDS layouts, with edges routed around node obstacles. The rendering pipeline builds a hierarchy of tiles—analogous to map tiles—so that at any zoom level only a bounded number of entities

[1] `@msagl/core`, `@msagl/drawing`, `@msagl/parser`, `@msagl/renderer-webgl` — all available on `https://www.npmjs.com/`.

[2] `https://github.com/microsoft/msagljs`

© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0
34th International Symposium on Graph Drawing and Network Visualization (GD 2026).
Editors: TBD; Article No. 0; pp. 0:1–0:13
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

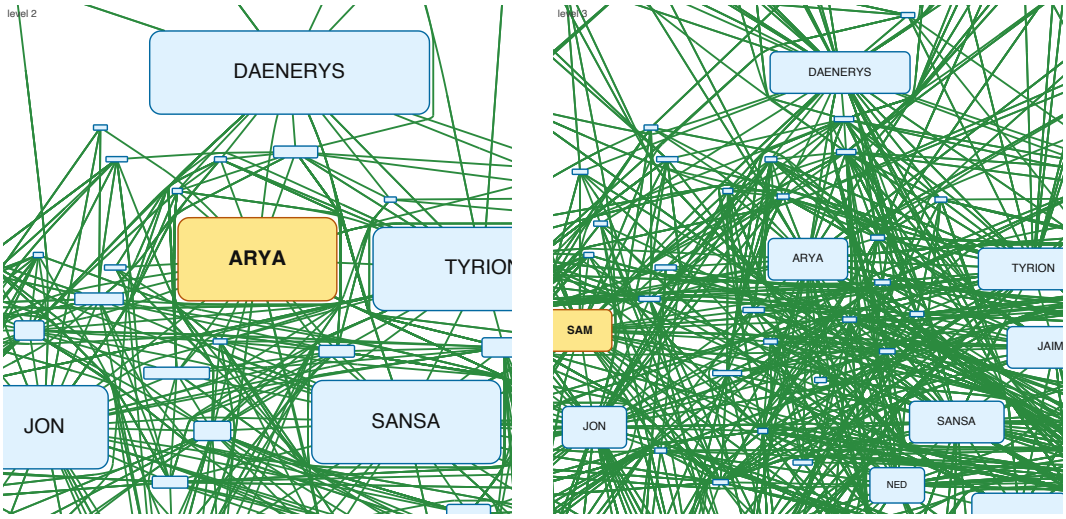


**Figure 1** Overview of the per-level-graph tiling scheme presented in Section 4, on the Game of Thrones graph. Two adjacent pyramid levels are shown, cropped to the same window; the coarser level is on the left, the finer one on the right. For each level a new graph is assembled from a PageRank-ranked subset of the nodes, and the top-ranked landmark, highlighted in amber, enters that level enlarged by a factor of two in each dimension. Remaining survivors, shown in light blue, are kept at a node-specific scale $s_v \in [0.25, 2]$ that shrinks with rank so that they do not overlap. Edges are rerouted per level with a sleeve-hint A* built from the finer-level route, which keeps the coarse shape close to the fine one and yields smooth transitions as the user pans and zooms.

are drawn. Live demos are available online: a WebGL renderer for large graphs[3] and an SVG renderer for smaller, editable graphs[4].

This paper makes two main contributions:

1. A *corridor routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation. The method introduces a *portal graph* that connects node obstacles through free-space CDT triangles, and accelerates shortest-path queries using Contraction Hierarchies [26] and Hub Labeling [14].
2. A *tiling scheme* for large graph visualization that limits visible entities per tile, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.

## 2 Related Work

#### 2.0.0.1 Graph visualization tools.

Graphviz [25, 6] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [9]; it does not support tiling or WebGL rendering. Our library builds on the earlier MSAGL desktop layout engine [39]. On the web, Sigma.js [10] and Cytoscape.js [23] provide interactive graph rendering, but rely on straight-line or generic spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles. ReGraph [8] uses WebGL to render large

[3] `https://microsoft.github.io/msagljs/renderer-webgl/index.html`
[4] `https://microsoft.github.io/msagljs/renderer-svg/index.html`

graphs with straight-line edges and supports interactive cluster expansion but not tiling. Cosmograph [3] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [38] supports tiling and level-of-detail for large graphs but runs only on Windows and does not optimize edge routes. Cornac [41] handles huge graphs with tiles and edge bundling [36, 34] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [13], which organizes nodes into a hierarchical cluster tree, and topological fisheye views [24], which distort a multilevel layout [29, 35] under the focus; neither targets browser-side tile pyramids or per-level edge rerouting. Hierarchical edge bundling [33] and force-directed bundling [34] are complementary clutter-reduction techniques applied *after* edges have been routed.

#### 2.0.0.2 Shortest paths in polygonal domains.

Our corridor algorithm walks the dual of a Constrained Delaunay Triangulation [42] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [37, 18, 28, 5]; optimal shortest-path algorithms in polygonal domains with holes [32] use related ideas but build a full shortest-path map, which we avoid.

#### 2.0.0.3 Edge routing.

Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [20] but requires a simple containing polygon. yWorks [12] offers "Organic edge routing" based on force-directed simulation. The approach of Dwyer and Nachmanson [22] routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid framework of Wybrow, Marriott and Stuckey performs obstacle-avoiding connector routing with incremental updates [43]. Our corridor method eliminates the spanner entirely and routes directly on the CDT dual graph, and is, to our knowledge, the first routing algorithm integrated with a browser-side tile pyramid that reroutes edges per level.

#### 2.0.0.4 Shortest-path speedup.

We adapt speedup techniques from road-network routing [15]. Contraction Hierarchies (CH) [26] preprocess a graph by iteratively contracting nodes and adding shortcut edges, enabling bidirectional Dijkstra queries that explore only a small fraction of the graph. Hub Labeling (HL) [14] extends CH by precomputing, for each node, a compact set of *hub* nodes and distances, so that shortest-path queries reduce to a linear-time merge of two sorted label arrays. A* [30] and its landmark-based variant ALT [27] use goal-directed heuristics instead of a hierarchy; we use plain A* with a Euclidean heuristic for per-level rerouting, where the sleeve hint from the finer level plays the role of a landmark. These techniques were developed for road networks but apply naturally to our CDT dual graph, which has bounded degree: each triangle has at most three neighbors.

## 3 System Architecture

MSAGLJS is organized as a monorepo with five NPM packages:

`@msagl/core` — Graph data structures, layout algorithms (Sugiyama, force-directed, MDS), edge routing, CDT, tiling.

`@msagl/drawing` — Visual attributes (colors, shapes, labels) bridging the graph model and renderers.
`@msagl/parser` — Parsers for DOT and JSON graph formats.
`@msagl/renderer-svg` — SVG-based renderer for small graphs.
`@msagl/renderer-webgl` — WebGL renderer using deck.gl [4], with tiling support.

The layout pipeline proceeds in three phases:

1. **Layout.** Node positions are computed by the chosen algorithm. For large undirected graphs the default is force-directed layout (IPsepCola) initialized with Pivot MDS [17]; for directed graphs, Sugiyama layout is used.
2. **Edge routing.** Edges are routed around node obstacles. The routing mode is configurable; this paper focuses on *corridor routing* (Section 5).
3. **Tiling.** A tile hierarchy is built for level-of-detail rendering (Section 4).

The WebGL renderer (Section 9) consumes the tile hierarchy and renders the appropriate level of detail based on the current viewport.

# 4 Tiling

To visualize large graphs interactively, MSAGLJS builds a pyramid of tiles analogous to those used in online maps. Levels are numbered $0, 1, \ldots, Z$ from coarsest to finest; the finest level $Z$ shows the full graph produced by the layout and the corridor router presented in Section 5. The construction proceeds in three phases.

## 4.1 Tile Hierarchy Construction

A tile is a pair $(\mathit{rect}, \mathit{tiledata})$, where *tiledata* is a set of *tile elements*: nodes, edge labels, edge arrowheads, and *edge clips*. An edge clip is a pair $(e, p)$, with $p$ a continuous piece of the edge curve $c_e$ confined to *rect*.

The single tile at level $Z$ is obtained by clipping every element to the graph's bounding box. To build level $z-1$ from level $z$, each tile is split into four equal sub-tiles; nodes, labels, and arrowheads are assigned to sub-tiles by bounding-box intersection, and each edge clip is cut at the tile midlines, producing sub-clips confined to individual sub-tiles. We maintain invariant $\mathcal{F}$: (a) every clip stays inside its tile's rectangle, crossing the boundary only at its endpoints; (b) the union of all clips for edge $e$ inside a tile equals $c_e \cap \mathit{rect}$. Subdivision of a level stops as soon as either of the following holds: (i) every tile at that level already contains at most $\mathcal{C}$ elements, with $\mathcal{C}$ default 5 000, so no tile needs splitting; or (ii) the tile width and height have both dropped below ten times the average node width and height, respectively, so further splitting would produce tiles smaller than the nodes they must display.

## 4.2 Per-Level Graph and PageRank-Guided Filtering

To keep coarser levels readable, each level $z < Z$ is assigned its own *level graph* $G_z$: a subset $V_z \subseteq V$ of active nodes together with every edge of $G$ whose endpoints both lie in $V_z$. We first compute PageRank [40] on $G$ once and sort nodes in decreasing rank.

Starting from $V_Z = V$, we build $V_{z-1}$ greedily. Let $r(v)$ be the rank of $v$ in $V_z$, so that rank 1 denotes the highest PageRank in $V_z$. The top-ranked node $v^\star$ is always accepted and *enlarged by a factor of two in each dimension*; lower-ranked nodes inherit a scale that is monotonically non-increasing in rank. We walk the top half of $V_z$ in rank order and, for each candidate $v$, pick the largest scale $s_v \in [s_{\min}, 2]$, decreasing geometrically, whose inflated

bounding box, padded by a margin $m = 3p + p_{\text{extra}}$ matching the obstacle inflation used by the router, does not intersect any previously accepted box. Scales below 1 are allowed, down to $s_{\min} = 0.25$, so that crowded lower-ranked nodes can still be shown at a reduced size rather than being dropped; a candidate whose box overlaps even at $s_{\min}$ is skipped. The accepted nodes form $V_{z-1}$ and the chosen scales are stored per level. This produces the intended map-like effect: a small number of big, highly ranked landmarks at the coarsest zoom, and progressively more detail as the user zooms in, with no lower-ranked node ever drawn larger than a higher-ranked one.

## 4.3 Stable Per-Level Rerouting with a Routing Hint

Because $G_{z-1}$ has fewer obstacles than $G_z$, and different ones on account of the enlargement of $v^\star$, edges of $G_{z-1}$ must be rerouted: the finest-level curves would otherwise pierce the enlarged landmark boxes. We rerun the corridor router (Section 5) on $G_z \setminus G_{z-1}$'s active edges, but with two modifications that secure visual stability during zoom.

#### 4.3.0.1 Per-level CDT.

For level $z-1$ we build a dedicated CDT from the inflated polylines of $V_{z-1}$ only, using each node's level-$z-1$ scale. The highest-ranked landmark thus enters the triangulation as an obstacle that is twice its finest-level size in each dimension, so routes naturally bend around it.

#### 4.3.0.2 Routing hint.

For every edge $e \in E_{z-1}$ we already have a route $c_e$ computed at the finer level $z$. We sample $c_e$, locate the CDT triangles it visits at level $z-1$, and expand this sequence by one neighbor hop to obtain a triangle *sleeve* $S_e$. Instead of the per-source Dijkstra trees described in Section 5.4, which are efficient when many edges share a source but ignore the previous level's shape, we run a per-edge **A*** on the dual of the level-$z-1$ CDT, *restricted to* $S_e$, with the source and target obstacle triangles always admissible. The hint keeps the coarse-level route topologically and geometrically close to the fine-level one, so that as the user zooms out an edge deforms smoothly rather than jumping to a different side of a landmark. If the masked A* fails, for instance when the target now lies inside the inflated landmark, we fall back to unconstrained A* on the whole dual, and finally to A* with the obstacle-interior guard disabled; all three tiers share the same pre-allocated open-set arrays.

To summarize the division of labor: **Dijkstra trees**, Section 5.4, drive the *finest-level* routing, where there is no hint and many edges share sources; **A*** drives the *per-level rerouting* for $z < Z$, where each edge carries its own sleeve hint from the finer level and independent searches are required. Figure 1 on the first page shows two adjacent pyramid levels built by this procedure. Once $E_{z-1}$ has been rerouted, the clip invariant $\mathcal{F}$ is reestablished by recomputing the clips of the affected edges inside every level-$(z-1)$ tile.

# 5 Corridor Routing on the CDT

We describe an edge routing method that routes edges directly on the dual graph of a Constrained Delaunay Triangulation, without requiring a spanner graph. The method finds a *corridor*—a sequence of adjacent CDT triangles from source to target—and applies the funnel algorithm to produce the shortest path within it.

## 5.1 CDT Construction and the Dual Graph

Given the graph layout, each node is covered by a padded obstacle polygon. We compute a Constrained Delaunay Triangulation $\mathcal{T}$ on the set $\mathcal{O}$ of these obstacle polygons [19, 21]. The *dual graph* $D$ of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing a side. A triangle is *free-space* if it is not contained in any obstacle interior (i.e., not all three of its vertices belong to the same obstacle). The dual-graph edge weight is the Euclidean distance between triangle centroids.

## 5.2 A* Search on the Dual Graph

For each graph edge from source $s$ to target $t$, we locate the containing triangles and run A* [30] on the dual graph $D$. The heuristic is the straight-line distance from the current centroid to $t$. Triangles inside obstacles (other than the source and target obstacles) are excluded from the search. The resulting corridor is passed to the funnel algorithm [18, 31], which computes the shortest path through the corridor's sequence of diagonals.

## 5.3 Vertex Collapse at Source and Target

The corridor passes through triangles whose vertices lie on the padded obstacle boundaries. When these vertices appear as diagonal endpoints, the funnel algorithm produces sharp zigzag turns near each endpoint. We eliminate these artifacts by *collapsing* obstacle vertices: any diagonal endpoint belonging to the source obstacle is replaced with the source center $s$, and similarly for the target. Degenerate diagonals (both endpoints collapsed to the same point) are removed. The left/right orientation of each diagonal is preserved from the uncollapsed geometry.

After collapse, the funnel sees a wide opening at each endpoint and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves.

## 5.4 Batched Routing with Dijkstra Trees

When a node has many incident edges, running independent A* searches is wasteful. We group edges by source node and run a single Dijkstra computation per source on the dual graph, expanding until all target triangles are reached. Corridors are recovered by following parent pointers. This reduces the number of searches from $m$ (edges) to at most $n$ (nodes). On the Game of Thrones social network (407 nodes, 2 639 edges, 331 distinct sources), the Dijkstra tree approach reduces routing time by 30%.

# 6 The Portal Graph

The batched Dijkstra approach of Section 5.4 still requires exploring much of the CDT for each source node. To enable faster queries, we introduce the *portal graph*—an abstraction that separates the free-space routing problem from the local obstacle traversal.

## 6.1 Portal Triangles

For each node obstacle, we identify its *portal triangles*: the free-space CDT triangles that share an edge with the obstacle boundary. Formally, a triangle $t$ is a portal of obstacle $O$ if $t$ is free-space and at least two of its three CDT sites belong to $O$'s boundary polyline.

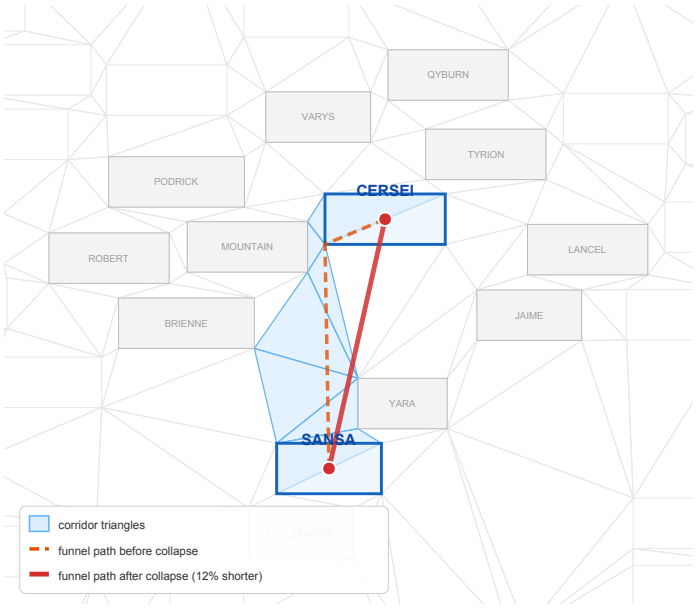


**Figure 2** Effect of vertex collapse on the CERSEI–SANSA edge. Light blue: corridor triangles. Orange dashed: funnel path before collapse (3 waypoints). Red solid: funnel path after collapse (2 waypoints, 12% shorter). Collapsing obstacle vertices to the node center widens the funnel opening and removes the detour.

Portals serve as entry and exit points: any shortest path from the interior of $O$ to free space must pass through a portal triangle.

## 6.2 Structure of the Portal Graph

The portal graph $G_P$ is defined as follows:

- **Nodes.** For each graph node $v$, the center point $c_v$ and all portal triangles $\{p_1^v, p_2^v, \ldots\}$ of $v$'s obstacle are nodes of $G_P$.
- **Interior edges.** Each center $c_v$ is connected to every portal $p_i^v$ of the same obstacle, with weight equal to the BFS distance through the obstacle's interior triangles.
- **Free-space edges.** Portal triangles of different obstacles are connected through the free-space CDT dual graph.

To route a graph edge from node $w$ to node $u$, we find the shortest path in $G_P$ from $c_w$ to $c_u$. This path decomposes into three segments:

1. An *interior path* from $c_w$ through $w$'s obstacle to some portal $p_i^w$.
2. A *free-space path* from $p_i^w$ to some portal $p_j^u$ of $u$.
3. An *interior path* from $p_j^u$ through $u$'s obstacle to $c_u$.

The free-space segment (2) is the bottleneck: it requires a shortest-path query on the free-space dual graph, which may contain thousands of triangles. Sections 7 and 8 show how to accelerate this query.

# 7 Contraction Hierarchies on the CDT Dual

To accelerate free-space queries in the portal graph, we preprocess the free-space CDT dual graph using Contraction Hierarchies (CH) [26].

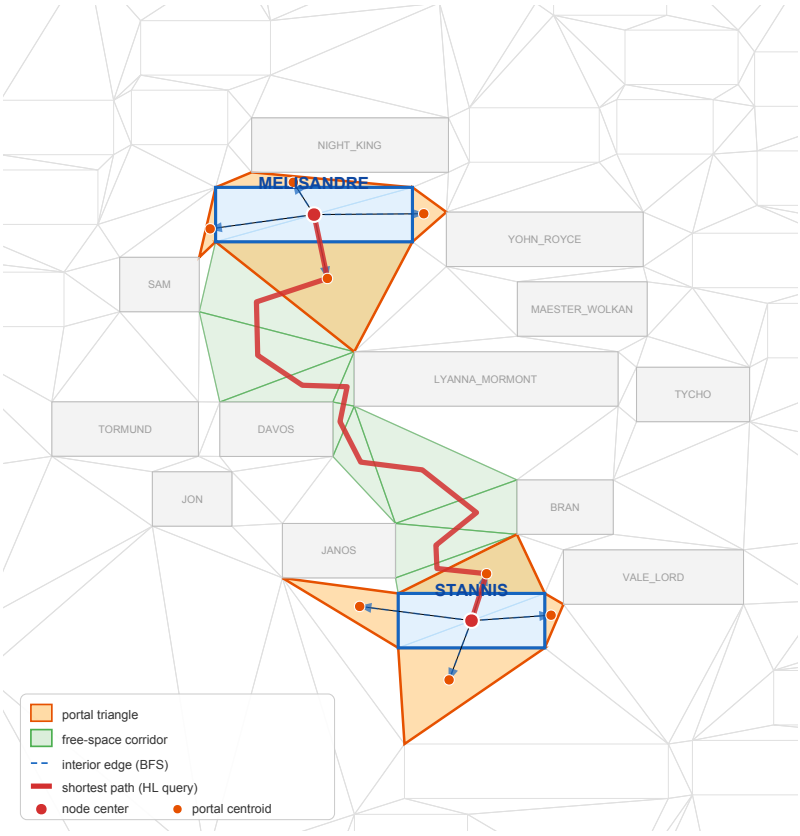


**Figure 3** The portal graph for the MELISANDRE–STANNIS edge on the Game of Thrones social network. Orange triangles are portals—free-space CDT triangles adjacent to the obstacle boundary. Blue dashed arrows show interior edges from node centers to portal centroids. Green-shaded triangles form the free-space corridor found via Hub Labeling. The thick red polyline shows the shortest path $c_{\text{MEL}} \to p_i \to \cdots \to p_j \to c_{\text{STAN}}$. Nearby node obstacles are shown in gray.

## 7.1 Preprocessing

The CH is built on the free-space dual graph: only triangles not contained in any obstacle interior are included. Each node (triangle) has at most three neighbors (since each triangle has three sides), so the graph is sparse throughout preprocessing.

Nodes are contracted in order of increasing *edge difference*: the number of shortcuts added minus the number of edges removed. When contracting a node $v$ with neighbors $a$ and $b$, if the shortest path from $a$ to $b$ passes through $v$, a *shortcut* arc is added from $a$ to $b$ with weight $w(a, v) + w(v, b)$. The shortcut stores a *midNode* pointer to $v$ for later path unpacking. Each contracted node receives a *rank* equal to its contraction order.

## 7.2 Bidirectional Query

Given source and target triangles, the CH query runs bidirectional Dijkstra where both the forward and backward searches follow only *upward arcs*—arcs to nodes of higher rank. The searches meet at some node $\mu$ of highest rank on the shortest path. The query time is $O(\log T)$ in practice, where $T$ is the number of free-space triangles, because the upward-only searches explore far fewer nodes than a standard Dijkstra.

## 7.3 Sleeve Recovery

The shortest path found by the CH query is a sequence of CH arcs, some of which are shortcuts. To obtain the corridor (sleeve) for the funnel algorithm, we recursively *unpack* each shortcut: an arc from $a$ to $b$ with midNode $v$ is replaced by the arcs $a \to v$ and $v \to b$, continuing until all arcs correspond to original CDT edges. The resulting sequence of CDT edges defines the corridor through which the funnel algorithm produces the final polyline path.

# 8 Hub Labeling for $O(|L|)$ Queries

While CH queries are fast, the per-query cost can be further reduced using Hub Labeling (HL) [14]. HL precomputes, for each node $v$, a *label* $L(v)$ consisting of pairs $(h, \delta(v, h))$ where $h$ is a *hub* node and $\delta(v, h)$ is the shortest-path distance from $v$ to $h$. Labels satisfy the *cover property*: for every pair $(s, t)$, the shortest $s$–$t$ path passes through some hub in $L(s) \cap L(t)$.

## 8.1 Label Construction

Labels are computed by processing nodes in decreasing CH rank order. For each node $v$, a forward CH search (following only upward arcs) is run from $v$. The search applies *stall-on-demand* pruning [14]: a node $w$ is stalled if some already-settled neighbor provides a shorter path to $w$, indicating that $w$'s optimal label entry will come from a higher-ranked node. The surviving search entries form $L(v)$.

Each label is stored as three parallel arrays—hub indices, distances, and parent pointers—sorted by hub index to enable efficient merging.

## 8.2 $O(|L|)$ Query

Given source $s$ and target $t$, the query merges their sorted label arrays with a two-pointer sweep:

```
i_s ← 0, i_t ← 0, d* ← ∞
while i_s < |L(s)| and i_t < |L(t)| do
    if L(s)[i_s].hub = L(t)[i_t].hub then
        d* ← min(d*, L(s)[i_s].dist + L(t)[i_t].dist)
        i_s ← i_s + 1; i_t ← i_t + 1
    else if L(s)[i_s].hub < L(t)[i_t].hub then
        i_s ← i_s + 1
    else
        i_t ← i_t + 1
    end if
end while
return d*
```

The query runs in $O(|L|)$ time, where $|L|$ is the average label size. For CDT dual graphs with bounded degree, labels are compact in practice.

## 8.3 Routing with Hub Labels

The full routing pipeline using Hub Labels proceeds as follows:

1. Build the CDT on padded obstacle polylines and construct the triangle index.
2. Build CH and HL on the *free-space* CDT dual graph (excluding obstacle interiors).

3. For each graph node, compute its portal triangles and map them to CH/HL indices.
4. For each graph edge $w \to u$:
   a. Query HL for all portal pairs $(p_i^w, p_j^u)$, computing the total distance as $\delta(c_w, p_i^w) + \mathrm{HL}(p_i^w, p_j^u) + \delta(p_j^u, c_u)$.
   b. Select the pair minimizing total distance.
   c. Recover the free-space sleeve from HL parent pointers and unpack CH shortcuts.
   d. Compute interior paths from centers to portals via BFS through obstacle triangles.
   e. Stitch the three sleeve segments (source interior, free-space, target interior).
   f. Apply vertex collapse (Section 5.3), then the funnel algorithm to produce the final polyline.

The CH/HL preprocessing is done once and amortized over all edges. For a graph with $m$ edges, $n$ nodes, and $T$ free-space triangles, the total routing cost is $O(T \log T)$ for preprocessing plus $O(m \cdot |L|)$ for all queries, compared to $O(m \cdot T \log T)$ for per-edge Dijkstra.

## 9 WebGL Rendering

The WebGL renderer uses deck.gl [4], a GPU-accelerated framework for large-scale data visualization. The renderer sets up an `OrthographicView` (no perspective distortion) and a `TileLayer` that dynamically loads tile data based on the current viewport.

#### 9.0.0.1 Tile resolution mapping.

The root tile size is rounded up to the nearest power of two. A *start zoom* level is computed so that the root tile maps to the standard 512-pixel tile size of deck.gl. The `TileLayer`'s `getTileData` callback translates deck.gl tile coordinates $(x, y, z)$ to MSAGLJS tile map coordinates, fetching the precomputed tile data.

#### 9.0.0.2 Rendering layers.

Each tile is rendered as a composite deck.gl layer containing:

- A **GeometryLayer** for node shapes (rectangles, ovals, diamonds).
- A **TextLayer** for node labels.
- A custom **CurveLayer** for edge paths, supporting cubic Bézier curves, arcs, and line segments via GPU-side interpolation. Curve tessellation resolution scales with zoom: at zoom level $z$, the resolution is $2^{z-2}$ segments per curve span.
- An **IconLayer** for edge arrowheads.
- A **TextLayer** for edge labels.

#### 9.0.0.3 Style-based filtering.

A declarative style specification allows per-layer control of visibility ranges (`minZoom`, `maxZoom`), colors, opacity, sizes, and filters. Styles can reference the node's PageRank to, e.g., show labels only for important nodes at medium zoom levels.

#### 9.0.0.4 Interaction.

The renderer supports pan, zoom, hover highlighting, and click selection. Hovering an edge highlights it and its incident nodes. The `zoomTo` API animates the viewport to a target rectangle with smooth transitions.

| Graph | Nodes | Edges | CDT △ | Free △ | Portals | Avg $\lvert L\rvert$ | Max $\lvert L\rvert$ | Dijkstra (ms) | HL (ms) |
|---|---|---|---|---|---|---|---|---|---|
| GoT [16] | 407 | 2 639 | 3 258 | 2 444 | 1 628 | 36.6 | 59 | 277 | 313 |
| b103 [2] | 944 | 2 438 | 7 554 | 5 666 | 3 776 | 57.9 | 89 | 754 | 882 |
| b100 [1] | 1 463 | 5 806 | 11 706 | 8 780 | 5 852 | 68.2 | 103 | 2 120 | 2 755 |
| composers [11] | 3 405 | 13 832 | 27 242 | 20 432 | 13 620 | 94.9 | 143 | 9 335 | 11 699 |
| Gnutella04 [7] | 10 876 | 39 994 | 87 010 | 65 258 | 43 504 | 163.5 | 255 | 93 760 | 90 746 |

**Table 1** Corridor routing benchmarks. Dijkstra: per-source Dijkstra trees on the CDT dual. HL: Hub Labeling with CH preprocessing (includes the CH+HL build time of 76–30 480 ms). Avg $|L|$ and Max $|L|$ are the average and maximum hub-label sizes across all free-space triangles.

## 10 Experimental Results

We evaluate the corridor routing method on benchmark graphs from the Game of Thrones social network [16], the GD 2011 contest [11], the Graphviz test suite [2, 1], and the Gnutella P2P network [7]. All experiments run in Node.js on a laptop with an Apple M1 processor. Layout uses Pivot MDS; timings include CDT construction, routing, and curve smoothing.

Table 1 compares the Dijkstra-tree corridor router (Section 5.4) against the Hub-Labeling router (Section 8.3). For each graph, we report the number of CDT triangles, free-space triangles (which form the CH/HL graph), the total number of portal triangles across all nodes, the average and maximum hub-label sizes $|L|$, and the total routing time.

The Dijkstra-tree approach is faster on the smaller graphs because the CH+HL preprocessing cost is not amortized when routing is done only once. However, on Gnutella04 (10 876 nodes, 39 994 edges), the HL router matches Dijkstra at 91 s vs. 94 s, since the per-query savings from $O(|L|)$ queries offset the 30 s preprocessing cost. The HL approach would become more advantageous in scenarios requiring repeated re-routing (e.g., interactive editing or level-of-detail re-routing during tiling), where the preprocessing cost is paid once.

The average label size grows sub-linearly with the number of free-space triangles: $|L| \approx 37$ for 2 444 triangles and $|L| \approx 164$ for 65 258 triangles. The maximum label size remains moderate (59–255), confirming that CDT dual graphs—with their bounded degree of 3—produce compact hub labels.

## 11 Conclusion

We presented MSAGLJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The corridor routing method routes edges directly on the CDT dual graph using a portal graph abstraction, accelerated by Contraction Hierarchies and Hub Labeling. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge simplification. The WebGL renderer, built on deck.gl, provides smooth pan-and-zoom interaction.

MSAGLJS is available at `https://github.com/microsoft/msagljs` under the MIT license.

## References


1. b100. `https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b100.dot`.
2. b103. `https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b103.dot`.
3. Cosmograph. `https://cosmograph.app`.
4. deck.gl: Large-scale WebGL-powered data visualization. `https://deck.gl/`.

5 Funnel algorithm. https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf.

6 Graphviz. http://www.graphviz.org/.

7 p2p-gnutella04. https://snap.stanford.edu/data/p2p-Gnutella04.html.

8 Regraph. https://cambridge-intelligence.com/regraph/.

9 sfdp. https://graphviz.org/docs/layouts/sfdp/.

10 Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. https://www.sigmajs.org/.

11 Skewed. http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml.

12 yworks. https://yworks.com/products/yed.

13 James Abello, Frank Van Ham, and Neeraj Krishnan. ASK-GraphView: A large scale graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*, volume 12, pages 669–676. IEEE, 2006.

14 Ittai Abraham, Daniel Delling, Andrew V Goldberg, and Renato F Werneck. A hub-based labeling algorithm for shortest paths in road networks. In *International Symposium on Experimental Algorithms*, pages 230–241. Springer, 2011.

15 Hannah Bast, Daniel Delling, Andrew V. Goldberg, Matthias Müller-Hannemann, Thomas Pajor, Peter Sanders, Dorothea Wagner, and Renato F. Werneck. Route planning in transportation networks. In *Algorithm Engineering: Selected Results and Surveys*, volume 9220 of *Lecture Notes in Computer Science*, pages 19–80. Springer, 2016.

16 Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages 201–225. Routledge, 2018.

17 Ulrik Brandes and Christian Pich. Eigensolver methods for progressive multidimensional scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe, Germany, September 18-20, 2006. Revised Papers 14*, pages 42–53. Springer, 2007.

18 Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.

19 Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennyka Nauk*, 7(1):793–800, 1934.

20 David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD'97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.

21 Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008.

22 Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing: 17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised Papers 17*, pages 147–158. Springer, 2010.

23 Max Franz, Christian T. Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D. Bader. Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–311, 2016.

24 Emden R. Gansner, Yehuda Koren, and Stephen North. Topological fisheye views for visualizing large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468, 2005.

25 Emden R. Gansner and Stephen C. North. An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233, 2000.

26 Robert Geisberger, Peter Sanders, Dominik Schultes, and Christian Vetter. Exact routing in large road networks using contraction hierarchies. *Transportation Science*, 46(3):388–404, 2012.

27 Andrew V. Goldberg and Chris Harrelson. Computing the shortest path: A* search meets graph theory. *SODA*, pages 156–165, 2005.

**28** Leonidas Guibas, John Hershberger, Daniel Leven, Micha Sharir, and Robert E. Tarjan. Linear-time algorithms for visibility and shortest path problems inside triangulated simple polygons. *Algorithmica*, 2(1–4):209–233, 1987.

**29** David Harel and Yehuda Koren. A fast multi-scale method for drawing large graphs. In *Journal of Graph Algorithms and Applications*, volume 6, pages 179–202, 2002.

**30** Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. `doi:10.1109/TSSC.1968.300136`.

**31** John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy class. *Computational geometry*, 4(2):63–97, 1994.

**32** John Hershberger and Subhash Suri. An optimal algorithm for Euclidean shortest paths in the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999.

**33** Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.

**34** Danny Holten and Jarke J. Van Wijk. Force-directed edge bundling for graph visualization. In *Computer Graphics Forum*, volume 28, pages 983–990. Wiley Online Library, 2009.

**35** Yifan Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–71, 2005.

**36** Christophe Hurter, Ozan Ersoy, and Alexandru Telea. Graph bundling by kernel density estimation. In *Computer graphics forum*, volume 31, pages 865–874. Wiley Online Library, 2012.

**37** D. T. Lee and Franco P. Preparata. Euclidean shortest paths in the presence of rectilinear barriers. *Networks*, 14(3):393–410, 1984.

**38** Lev Nachmanson, Roman Prutkin, Bongshin Lee, Nathalie Henry Riche, Alexander E Holroyd, and Xiaoji Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA, September 24-26, 2015, Revised Selected Papers 23*, pages 3–15. Springer, 2015.

**39** Lev Nachmanson, George G. Robertson, and Bongshin Lee. Drawing graphs with GLEE. In *Graph Drawing: 15th International Symposium, GD 2007*, pages 389–394. Springer, 2008.

**40** Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999.

**41** Alexandre Perrot and David Auber. Cornac: Tackling huge graph visualization with big data infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018.

**42** Jonathan Richard Shewchuk. Delaunay refinement algorithms for triangular mesh generation. *Computational Geometry*, 22(1–3):21–74, 2002.

**43** Michael Wybrow, Kim Marriott, and Peter J. Stuckey. Orthogonal connector routing. *Lecture Notes in Computer Science*, 5849:219–231, 2010.